



PRAKTIKUM SISTEM BASIS DATA

Nama	Putri Kiara Salsabila Arief	No. Modul	6
NPM	2306250743	Tipe	Tugas Tambahan

1. Melanjutkan backend sesuai dokumentasi.
2. Response.

Method	Endpoint	Success	Failed
transaction			
GET	/transaction	<pre>1 { 2 "success": true, 3 "message": "Transactions found", 4 "payload": [5 { 6 "id": "d927367-b753-4b21-ab14-b14802655808", 7 "user_id": "b0d0609f-a160-4e7b-a896-e1ac2c2e3aba", 8 "item_id": "07600876-7e4b-4b61-9b93-376a8236898f", 9 "quantity": 1, 10 "total": 100000, 11 "status": "paid", 12 "created_at": "2025-04-10T04:54:40.882Z", 13 "user": { 14 "id": "b0d0609f-a160-4e7b-a896-e1ac2c2e3aba", 15 "name": "Minji NewJeans", 16 "email": "minji.newjeans@example.com", 17 "password": "S2nS185NHCI3BoddpJ7RvP3d_XvWuZID4UjdxewrtaCnZCv026JEzLxam", 18 "balance": 100000, 19 "created_at": "2025-03-19T05:37:43.51982Z" 20 }, 21 "item": { 22 "id": "07600876-7e4b-4b61-9b93-376a8236898f", 23 "name": "Lanyard updated", 24 "price": 100000, 25 "store_id": "f6d59b64-2c44-4921-a365-b21f9db85334", 26 "image_url": "https://res.cloudinary.com/dsibjzsub/image/upload/v1742188675/czwhkbsz5vrt6lbyiep.jpg", 27 "stock": 5, 28 "created_at": "2025-03-16T06:18:37.636948" 29 } 30 }, 31 { 32 "id": "6325089e-4394-4e43-bce7-6439cc054f2", 33 "user_id": "a615c4dc-998e-4d87-b080-c87c28a8507", 34 "item_id": "07600876-7e4b-4b61-9b93-376a8236898f", 35 "quantity": 2, 36 "total": 200000, 37 "status": "pending", 38 "created_at": "2025-04-10T09:21:08.056Z", 39 "user": { 40 "id": "60a82000-cdc0-450f-bf06-130596dd594", 41 "name": "Kiaxa Asief", 42 "email": "kiaxaasief@example.com", 43 "password": "S2nS185NHCI3BoddpJ7RvP3d_XvWuZID4UjdxewrtaCnZCv026JEzLxam", 44 "balance": 200000, 45 "created_at": "2025-04-10T16:08:24.69588Z" 46 }, 47 "item": { 48 "id": "07600876-7e4b-4b61-9b93-376a8236898f", 49 "name": "Lanyard updated", 50 "price": 100000, 51 "store_id": "f6d59b64-2c44-4921-a365-b21f9db85334", 52 "image_url": "https://res.cloudinary.com/dsibjzsub/image/upload/v1742188675/czwhkbsz5vrt6lbyiep.jpg", 53 "stock": 5, 54 "created_at": "2025-03-16T06:18:37.636948" 55 } 56 } 57], 58 { 59 "id": "6325089e-4394-4e43-bce7-6439cc054f2", 60 "user_id": "a615c4dc-998e-4d87-b080-c87c28a8507", 61 "item_id": "07600876-7e4b-4b61-9b93-376a8236898f", 62 "quantity": 2, 63 "total": 200000, 64 "status": "pending", 65 "created_at": "2025-04-10T09:21:39.198Z", 66 "user": { 67 "id": "a615c4dc-998e-4d87-b080-c87c28a8507", 68 "name": "Hyemin NewJeans", 69 "email": "hyemin.nj@example.com", 70 "password": "S2nS185NHCI3BoddpJ7RvP3d_XvWuZID4UjdxewrtaCnZCv026JEzLxam", 71 "balance": 200000, 72 "created_at": "2025-04-10T16:09:29.158115Z" 73 }, 74 "item": { 75 "id": "07600876-7e4b-4b61-9b93-376a8236898f", 76 "name": "Lanyard updated", 77 "price": 100000, 78 "store_id": "f6d59b64-2c44-4921-a365-b21f9db85334", 79 "image_url": "https://res.cloudinary.com/dsibjzsub/image/upload/v1742188675/czwhkbsz5vrt6lbyiep.jpg", 80 "stock": 5, 81 "created_at": "2025-03-16T06:18:37.636948" 82 } 83 } 84]</pre>	<pre>1 <!DOCTYPE html> 2 <html lang="en"> 3 4 <head> 5 <meta charset="utf-8"> 6 <title>Error</title> 7 </head> 8 9 <body> 10 <pre>Cannot POST /transaction</pre> 11 </body> 12 13 </html></pre>

3. Meningkatkan kode express yang telah anda buat pada bagian scalability, security, dan error handling.

Menambahkan error handling.

Implementasi	Deskripsi
<pre> JS user.controller.js X src > controllers > JS user.controller.js > topUpUser > topUpUser 108 exports.topUpUser = async (req, res) => { 115 if (isNaN(amount) amount <= 0) { 116 return baseResponse(res, false, 400, "Amount must be larger than 0", null); 117 } </pre>	Memvalidasi input numerik (seperti amount) untuk menghindari error perhitungan dan mencegah eksploitasi (misalnya user top-up -100), sebagai bagian dari preventive error handling sebelum masuk ke logic atau database.

4. Scalability, Security, dan Error Handling selama Modul 5 dan 6.

Tipe	Implementasi	Deskripsi
Scalability	<pre> JS transaction.controller.js X src > controllers > JS transaction.controller.js > deleteTransaction > deleteTransaction > deletedTransaction 104 exports.getAllTransactions = async (req, res) => { 105 try { 106 const transactions = await transactionRepository.getAllTransactions(); 107 baseResponse(res, true, 200, "Transactions found", transactions); 108 } catch (error) { 109 baseResponse(res, false, 500, "Error retrieving transactions", error); 110 } 111 }; </pre>	Penggunaan modularisasi repository untuk menangani query database dilakukan agar kode lebih scalable saat aplikasi berkembang.
	<pre> JS user.controller.js X src > controllers > JS user.controller.js > updateUser > updateUser 3 const baseResponse = require("../utils/baseResponse.util"); </pre> (ada di semua controller)	Pemisahan response menjadi modul utilitas (baseResponse.util.js) memungkinkan reuse logic untuk response format ke semua controller.

Security

```
JS user.controller.js X
src > controllers > JS user.controller.js > registerUser > registerUser
5  const emailRegex = /^[^s@]+@[^s@]+\.[^s@]+$/; // email regex
6
7  exports.registerUser = async (req, res) => {
8    const { name, email, password } = req.body;
9    if (!name || !email || !password) {
10     return baseResponse(res, false, 400, "Name, email, and password are required", null);
11   }
12
13   if (!emailRegex.test(email)) {
14     return baseResponse(res, false, 400, "Invalid email format", null);
15   }

```

Pengecekan format email dengan regex dilakukan saat register dan update user. Tujuannya untuk mencegah user memasukkan input sembarangan yang bisa mengakibatkan injeksi data tidak valid, dan validasi keamanan.

```
JS user.controller.js X
src > controllers > JS user.controller.js > registerUser > registerUser
7  exports.registerUser = async (req, res) => {
23   const hashedPassword = await bcrypt.hash(password, 10);
JS user.controller.js X
src > controllers > JS user.controller.js > getUserByEmail > getUserByEmail
31  exports.loginUser = async (req, res) => {
45   const isValidPassword = await bcrypt.compare(password, user.password);

```

Menggunakan bcrypt untuk hash password saat register dan membandingkan saat login. Ini dilakukan untuk menghindari penyimpanan password dalam bentuk plaintext, sehingga walau database bocor, password tetap aman.

```
JS pg.database.js X
src > database > JS pg.database.js > ...
5  const pool = new Pool({
6    connectionString: process.env.PG_CONNECTION_STRING,
7    ssl: {
8      rejectUnauthorized: false,
9    },
10 });

```

Penggunaan environment variable (.env) untuk menyimpan credential seperti connection string PostgreSQL dan API key Cloudinary, membuat kode lebih aman. Untuk mencegah risiko bocornya data sensitif ke publik (misalnya saat push ke GitHub).

Error Handling

```
JS user.controller.js X
src > controllers > JS user.controller.js > updateUser > updateUser
60 exports.getUserByEmail = async (req, res) => {
61   const email = req.params.email;
62   try {
63     const user = await userRepository.getUserByEmail(email);
64     if (!user) {
65       return baseResponse(res, false, 404, "User not found", null);
66     }
67     baseResponse(res, true, 200, "User found", user);
68   } catch (error) {
69     baseResponse(res, false, 500, "Error retrieving user", error);
70   }
71 };
```

(ada di semua controller)

Semua controller dibungkus oleh try-catch dan diberi pengecekan data yang mungkin null. Tujuannya untuk menangani kemungkinan error koneksi database, data tidak ditemukan, dan error logic, jadi aplikasi bisa berjalan stabil tanpa crash.

```
JS baseResponse.util.js X
src > utils > JS baseResponse.util.js > ...
1 const baseResponse = (res, success, status, message, payload) => {
2   return res.status(status).json({
3     success,
4     message,
5     payload
6   });
7 }
```

Penggunaan fungsi utilitas baseResponse untuk semua response memungkinkan format responsif dan konsisten. Bisa mengurangi duplikasi kode dan menjaga struktur respons API tetap rapi dan seragam sekalipun banyak endpoint.

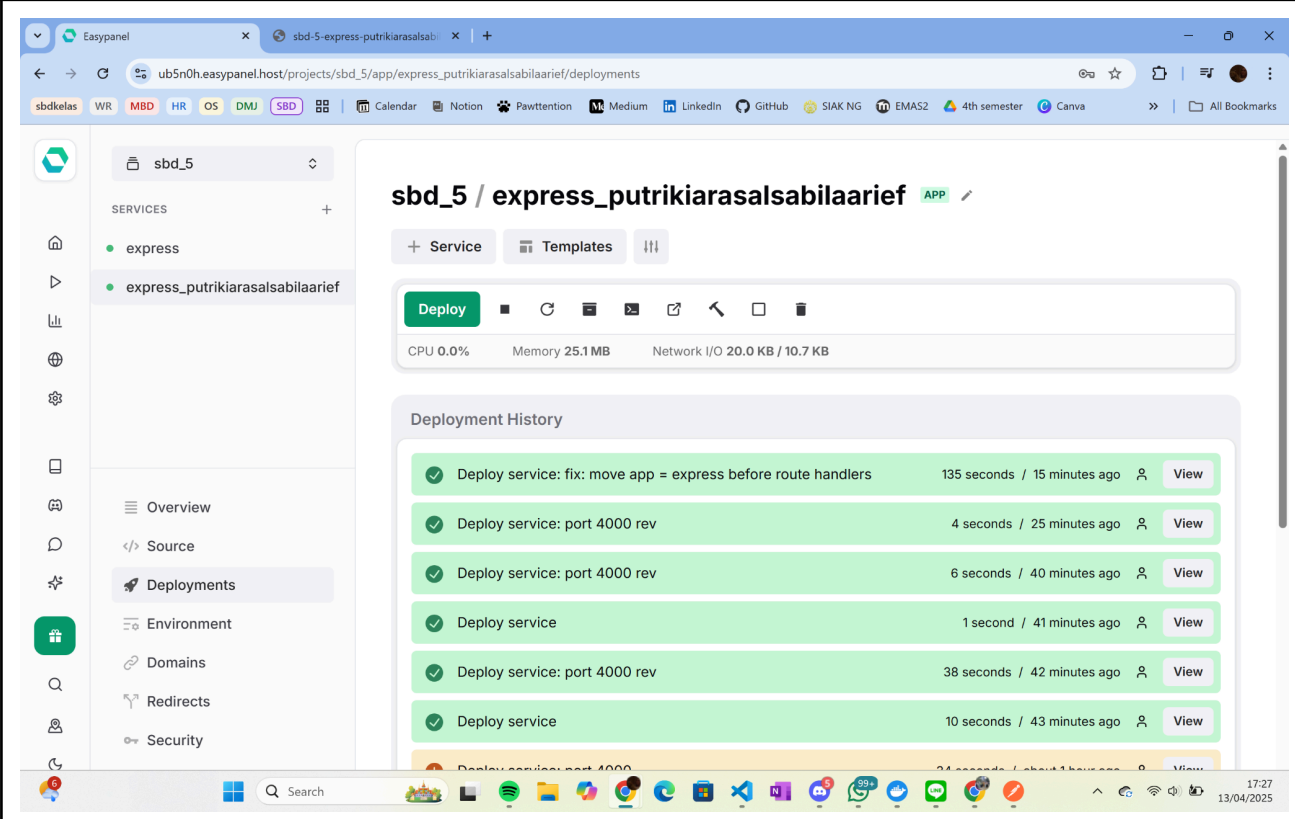
```
JS user.controller.js X
src > controllers > JS user.controller.js > topUpUser > topUpUser
108 exports.topUpUser = async (req, res) => {
115   if (isNaN(amount) || amount <= 0) {
116     return baseResponse(res, false, 400, "Amount must be larger than 0", null);
117   }
```

Memvalidasi input numerik (seperti amount) untuk menghindari error perhitungan dan mencegah eksploitasi (misalnya user top-up -100), sebagai bagian dari preventive error handling sebelum masuk ke logic atau database.

Stress Test

1. Deploy Backend.

Screenshot



The screenshot shows the Easypanel interface for managing services. The sidebar on the left contains navigation links: Overview, Source, Deployments (selected), Environment, Domains, Redirects, and Security. The main content area displays the service 'sbd_5 / express_putrikiarasalsabilaarief' with an 'APP' icon. Below the service name are tabs for '+ Service' and 'Templates'. A 'Deploy' button is prominently displayed. Resource usage is shown as CPU 0.0%, Memory 25.1 MB, and Network I/O 20.0 KB / 10.7 KB. The 'Deployment History' section lists six successful deployments with timestamps and 'View' links.

Deployment Action	Duration	Time Ago	Action
Deploy service: fix: move app = express before route handlers	135 seconds	15 minutes ago	View
Deploy service: port 4000 rev	4 seconds	25 minutes ago	View
Deploy service: port 4000 rev	6 seconds	40 minutes ago	View
Deploy service	1 second	41 minutes ago	View
Deploy service: port 4000 rev	38 seconds	42 minutes ago	View
Deploy service	10 seconds	43 minutes ago	View

Action

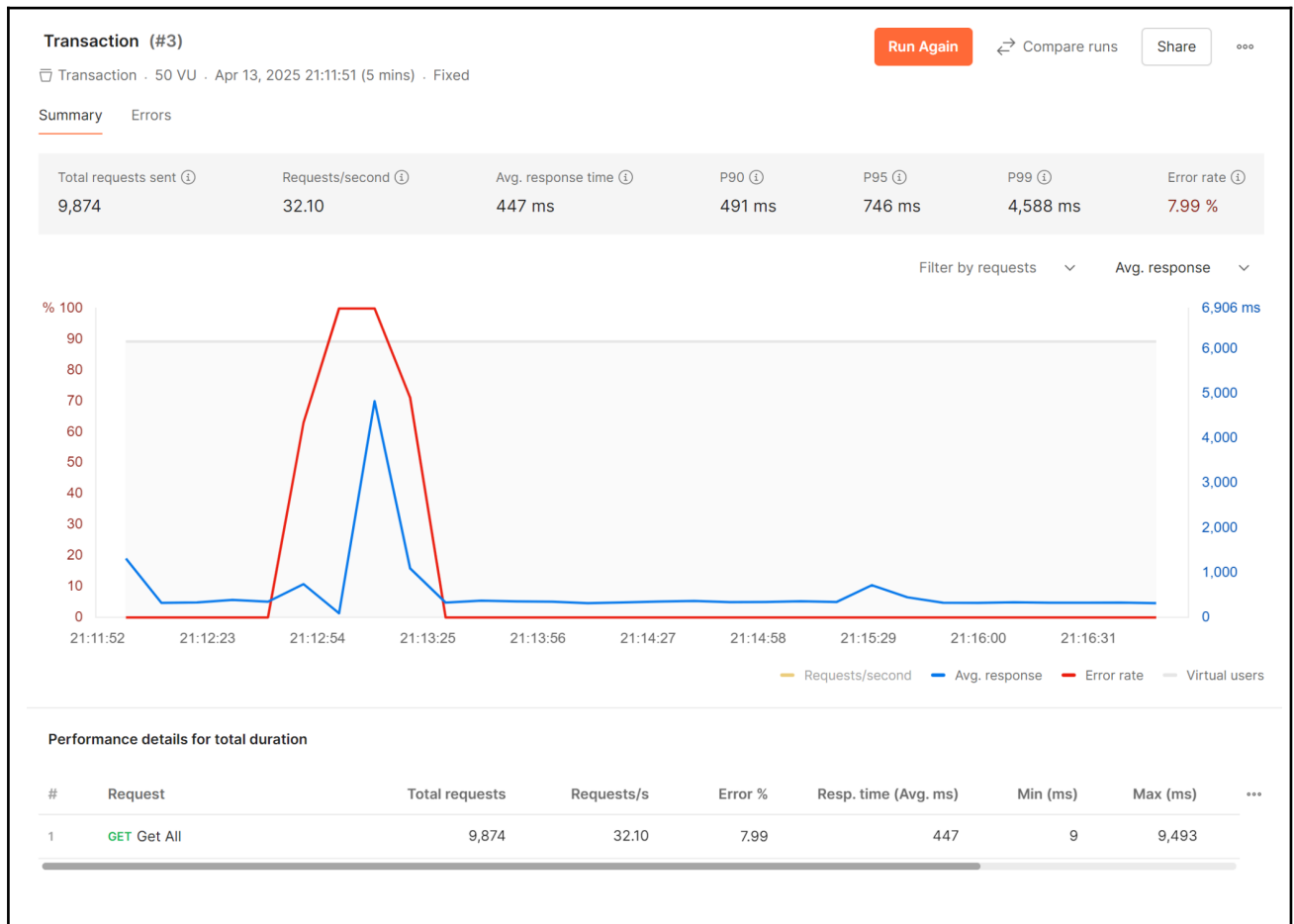
```
7.5.1 /src/app
#9 CACHED

#10 [6/6] COPY . .
#10 DONE 5.0s

#11 exporting to image
#11 exporting layers
#11 exporting layers 3.3s done
#11 writing image sha256:6572203fdbea18f0c0bb833378c87efd3e98d6e68dfa39b69086c81fe05af15d
#11 writing image sha256:6572203fdbea18f0c0bb833378c87efd3e98d6e68dfa39b69086c81fe05af15d done
#11 naming to docker.io/easypanel/sbd_5/express_putrikiarasalsabilaarief
#11 naming to docker.io/easypanel/sbd_5/express_putrikiarasalsabilaarief 0.1s done
#11 DONE 3.9s
#####
### Success
### Sun, 13 Apr 2025 10:14:43 GMT
#####
```

2. Memasukkan Data dummybonustutam.sql.
3. Stress Test dengan Error Rate <10%.

Screenshot



4. Penjelasan Bagian yang menyebabkan Stress Test berhasil.

Stress test berhasil karena aplikasi mampu menangani 50 virtual user selama 5 menit dengan error rate hanya 7,99%, di bawah batas 10%. Ini menunjukkan server cukup stabil dan respons responsnya sesuai dokumentasi, meskipun dalam kondisi akses ramai. Deployment pakai Docker dan koneksi database yang lancar juga bantu bikin sistem tetap jalan tanpa banyak error.